WONBET · FASE EXTRA — GUION DE VIDEO

Guion para el video de sustentación

~8 minutos, dividido en 5 bloques. Cada bloque dice quién habla, qué decir (en sus propias palabras, esto es guía no libreto para leer literal) y qué mostrar en pantalla — archivo y línea exacta en VS Code, o la acción exacta en la app ya desplegada.

DURACIÓN TOTAL	BLOQUES	CÓDIGO	DEMO EN VIVO
~8 min	5	VS Code	App en Railway

- Persona 1 — Alejandro
- Persona 2 — Julián
- Persona 3 — Santiago

Asignación por defecto, en el orden que me diste los nombres — cámbienla como quieran, la estructura de los bloques no depende de quién diga cada parte.

00 Intro

Los 3 · 0:30

QUÉ DECIR

> "Hola, somos Alejandro, Julián y Santiago. Este es WonBet, una plataforma de apuestas deportivas que construimos con arquitectura de microservicios."

> "En este video les mostramos cómo está armado el código y la aplicación ya funcionando, desplegada en internet — no solo en nuestras máquinas."

QUÉ MOSTRAR

PANTALLA El diagrama de arquitectura (la imagen/PDF que ya tienen) de fondo mientras hablan, o la carátula del repo.

01 Arquitectura general y el Gateway

Alejandro · 1:30

QUÉ DECIR

> "WonBet son 5 piezas independientes: un gateway, tres microservicios — usuarios, eventos y apuestas — y un frontend. Cada una corre por su cuenta, con su propio puerto y su propia base de datos SQLite."

> "El gateway es la única puerta de entrada, pero acá viene la decisión más importante de toda la arquitectura: el gateway **no verifica nada**. Es un proxy puro — recibe la petición y la reenvía tal cual al microservicio correcto, sin tocar el token."

> "Eso significa que la seguridad no depende de él: depende de que cada microservicio se cuide solo."

QUÉ MOSTRAR (VS CODE)

`EXPLORER` Abrir el árbol de carpetas: `gateway/`, `ms-usuarios/`, `ms-eventos/`, `ms-apuestas/`, `frontend/`.

`gateway/index.js` Línea `20`: el comentario "Este gateway NO verifica tokens". Línea `25`: la función `proxyA` — reenvía y ya. Líneas `40-46`: las tres rutas (`/api/usuarios`, `/api/eventos`, `/api/apuestas`).

02 Autenticación con JWT

Alejandro · 2:00

QUÉ DECIR

> "El JWT lo genera **ms-usuarios** cuando alguien hace login: valida la contraseña con bcrypt y firma un token con el correo y el rol del usuario."

> "Lo interesante es cómo lo verifican los otros dos servicios sin preguntarle nada a ms-usuarios: los tres comparten la misma clave secreta, hardcodeada a propósito en cada middleware. Con esa clave, cualquiera puede comprobar que la firma es válida."

> "También restringimos explícitamente el algoritmo a HS256 en la verificación — es una recomendación oficial de la librería para que nadie pueda forzar un algoritmo distinto."

QUÉ MOSTRAR (VS CODE)

`ms-usuarios/index.js` Línea `34`: `bcrypt.hash`. Línea `60`: el login. Línea `76`: `jwt.sign(...)` con `correo` y `rol`.

`ms-usuarios/middleware/auth.js` Línea `6`: el `JWT_SECRET` hardcodeado. Línea `26`: `jwt.verify(token, JWT_SECRET, { algorithms: ['HS256'] })`.

`TIP` Abrir el mismo archivo `middleware/auth.js` en `ms-eventos/` y `ms-apuestas/` un segundo, para que se vea que es idéntico en los tres.

03 Eventos, Apuestas y la comunicación interna

Julián 2:00

QUÉ DECIR

> "ms-eventos maneja el catálogo de eventos deportivos: nombre, cuota y si está abierto o cerrado. Todos sus endpoints piden el JWT, excepto uno."

> "Ese uno es `/internal/eventos/:id` — un endpoint sin autenticación, pensado solo para que otro microservicio lo consulte, nunca un usuario final."

> "Y el que lo consume es ms-apuestas: cuando alguien apuesta, le pega directo a ese endpoint interno de eventos — sin pasar por el gateway y sin token — para saber la cuota y si el evento sigue abierto. Si está cerrado, rechaza la apuesta. Si está abierto, calcula la ganancia potencial y la guarda."

QUÉ MOSTRAR (VS CODE)

`ms-eventos/index.js` Línea `21` : `GET /api/eventos` (con `verificarToken`). Línea `96` : `GET /internal/eventos/:id` (sin middleware — señalarlo).

`ms-apuestas/index.js` Línea `38` : `POST /api/apuestas` . Línea `64` : el `fetch(EVENTOS_URL + '/internal/eventos/...')` . Línea `77` : el `if (evento.estado !== 'abierto')` .

04 Demo en vivo: la app desplegada

Santiago 2:00

QUÉ DECIR

> "Todo lo que acabamos de ver no corre solo en nuestras laptops — está desplegado en Railway, como 5 servicios separados, igual que en el diagrama."

> "Vamos a registrarnos, iniciar sesión, ver los eventos disponibles, apostar en uno abierto, y después intentar apostar en uno que ya está cerrado para que vean el rechazo en vivo."

QUÉ MOSTRAR (NAVEGADOR, YA DESPLEGADO)

`ABRIR` `frontend-production-88c0.up.railway.app`

`HACER` 1. "Crear cuenta" con un correo nuevo. 2. Iniciar sesión. 3. Apostar en "Real Madrid vs Barcelona" o "Millonarios vs Nacional" (abiertos). 4. Intentar apostar en "Boca vs River" (cerrado) — mostrar que el botón está deshabilitado o el error si se fuerza por API.

`OPCIONAL` Un vistazo rápido al proyecto en el dashboard de Railway: los 5 servicios corriendo, para reforzar que es real.

05 Cierre

Los 3 · 0:30

QUÉ DECIR

> "Para resumir: WonBet muestra tres ideas de microservicios — un gateway que solo enruta, servicios que verifican su propio token con una clave compartida, y comunicación directa entre servicios para datos internos."

> "Lo mantuvimos simple a propósito: SQLite en vez de una base con servidor aparte, sin Docker, sin ORM — para poder explicarlo completo, de punta a punta. Gracias."

QUÉ MOSTRAR

PANTALLA Volver al diagrama de arquitectura como cierre visual.

Orden de grabación sugerido

BLOQUE	QUIÉN	DURA	ACUMULADO
00 · Intro	Los 3	0:30	0:30
01 · Arquitectura + Gateway	Alejandro	1:30	2:00
02 · JWT	Alejandro	2:00	4:00
03 · Eventos/Apuestas	Julián	2:00	6:00
04 · Demo desplegada	Santiago	2:00	8:00
05 · Cierre	Los 3	0:30	8:30

Si les queda corto o largo: el bloque 02 (JWT) es el más fácil de extender o recortar sin romper el hilo — es el corazón conceptual del proyecto, pero también el que más se presta a resumir si van con el tiempo justo.